Software Architecture and Design Specification

Project: Student Attendance Management System (SAMS)

Version: 1.2

Authors: Team 3 (Priyanka, Priyanka M Reddy, Priyanka N, Priyanka Sajjan)

Date: 04-10-2026

Status: Submitted for Review

Revision History

Version	Date	Author	Change Summary
1.0	29-09-2026	Team 3	Initial version submitted for review
1.1	30-09-2026	Team 3	Aligned with SRS v1.0 and STP v1.0: corrected traceability, added event-chain detail, idempotent retry handling, sliding-expiry login session, and documented SRS conflicts
1.2	04-10-2026	Team 3	Restructured to follow the course SAD template (Architecture / Design / Appendices); added Audience, Document Overview, Goals and Constraints, Stakeholders and Concerns, Technology Stack, Risks and Mitigations, UX Design, Open Issues and Next Steps, and Appendices

Approvals

Role	Name	Signature / Date
Group Lead	Priyanka N	
Course Coordinator		

Table of Contents

Revision History 1
Approvals 1
Table of Contents 1
1. Introduction 2
1.1 Purpose 3
1.2 Scope 3
1.3 Audience 3
1.4 Definitions 3
2. Document Overview 3
2.1 How to Use This Document 3
2.2 Related Documents 3
3. Architecture 4
3.1 Goals and Constraints 4
3.2 Stakeholders and Concerns 4
3.3 Component (UML) Diagram 4
3.4 Component Descriptions 5
3.5 Chosen Architecture Pattern and Rationale 6
3.6 Technology Stack and Data Stores 6
3.7 Risks and Mitigations 6
3.8 Traceability to Requirements 7
3.9 Security Architecture 7
4. Design 8
4.1 Design Overview 8
4.2 UML Sequence Diagrams 8
Mark Attendance (SAMS-F-008, F-009, NF-001, NF-002, SR-001, SR-004) 8
Recalculate Percentage and Low-Attendance Alert (SAMS-F-011, F-012, F-014) 9
Edit Attendance in Correction Window (SAMS-F-010, SR-002) 10
4.3 API Design 11
4.4 Error Handling, Logging and Monitoring 12
4.5 UX Design 13
4.6 Open Issues and Next Steps 13
5. Appendices 14
5.1 Glossary 14
5.2 References 14
5.3 Tools 14

1. Introduction

1.1 Purpose

This document specifies the architecture and detailed design of the Student Attendance Management System (SAMS). It explains how the system is structured, how its modules interact, and how each requirement in the Software Requirements Specification (SRS) is satisfied, so that developers can build the system and testers can verify it against the Software Test Plan (STP).

1.2 Scope

This document covers the architecture and design needed to deliver all functional, non-functional and security requirements in the SRS: authentication and role-based access, course and enrollment management, attendance marking and correction, percentage reporting, low-attendance notification, and leave requests. Biometric attendance capture, payroll integration, and mobile push notifications are out of scope, as stated in the SRS.

1.3 Audience

This document is written for the people who will build, test and review SAMS:

- **Developers:** module boundaries, API contracts, and data design needed to implement the system.
- **QA engineers:** the behavior each design element must satisfy, design test cases and compare against the STP.
- **Course coordinator / reviewer:** traceability from requirements to design, so coverage can be checked.
- **Future maintainers:** the reasoning behind each architectural decision, so later changes keep the same intent.

1.4 Definitions

Definitions follow SRS Section 1.4 (Student, Teacher, Admin, Attendance Record, Session/Period, Attendance Percentage, Defaulter, Correction Window, Roster). Terms used only in this document are listed in Appendix 5.1.

2. Document Overview

2.1 How to Use This Document

Section 3 (Architecture) describes the overall structure of SAMS: the chosen pattern, the components, how they are deployed, and how security is built in. It is the starting point for anyone checking whether the design meets the SRS at a system level.

Section 4 (Design) goes one level deeper: sequence diagrams for the main workflows, the REST API, error handling, and the data model. It is the reference for implementing and unit-testing individual modules.

Section 5 (Appendices) collects supporting material: a glossary, references, and the tools used to prepare this document.

2.2 Related Documents

Document	Description
Software Requirements Specification (SRS), SAMS, Team 3, v1.0, 04-09-2026	States the functional, non-functional and security requirements this design satisfies
Software Test Plan (STP), SAMS, Team 3, v1.0, 17-09-2026	Defines the test cases that verify this design once implemented
Requirements Traceability Matrix (RTM)	Included within this document as Section 3.8, rather than as a separate file

3. Architecture

3.1 Goals and Constraints

Goals

- Correct, role-appropriate access for Admin, Teacher, and Student (SRS Section 5).
- Fast, reliable attendance markings saved within 3 seconds for 60 students, with no data loss or duplication (SAMS-NF-001, NF-002).
- Support at least 100 concurrent users and stable operation over an 8-hour session (SAMS-NF-004, NF-006).
- Protect attendance and personal data from unauthorized access or change (SO-001, SO-002).

Constraints

- Must run in a standard web browser over HTTPS; no native client (SRS Section 2.4, 3.4).
- The final web framework and relational database are chosen at implementation time; the design stays in technology neutral (SRS Section 2.5).
- Institutional deployment: no dedicated operations team assumed, so the design favors a single deployable unit over a distributed one.

3.2 Stakeholders and Concerns

Stakeholder	Concern
Student	Accuracy of own attendance and percentage; privacy of personal data; timely alerts
Teacher	Fast marking for a full class; ability to correct a mistake; clear section reports
Admin	Control over accounts, courses and configuration; audit trail for disputes
Developers	Clear module boundaries and a stable API, so modules can be built and tested independently
QA / Testers	Behavior that can be verified against the STP without ambiguity
Course coordinator	Traceability from requirements to design, and confidence the architecture is sound

3.3 Component (UML) Diagram

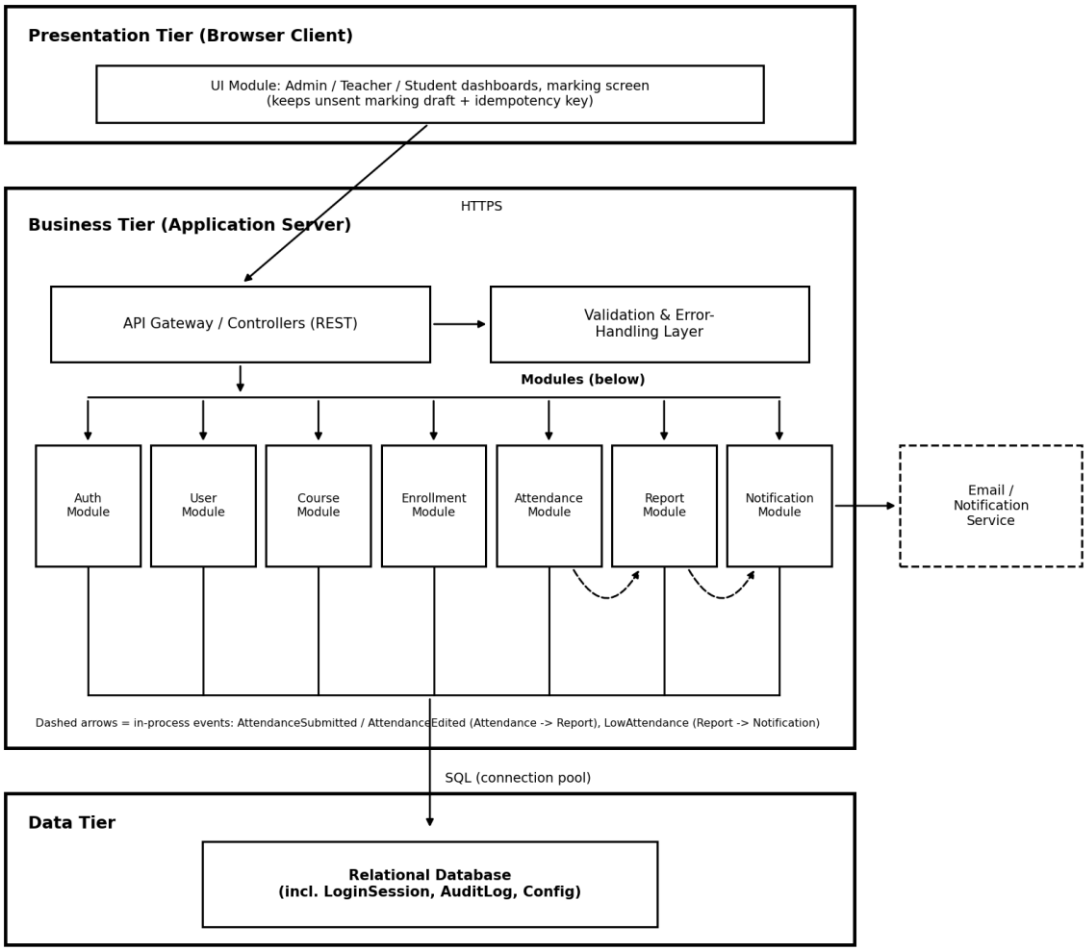


Figure 1: Component diagram of SAMS

3.4 Component Descriptions

Component	Responsibility	Depends on
UI Module	Role-specific dashboards and marking screens. Holds the unsent marking draft and the idempotency key so a failed or timed-out submission can be retried without re-entering data	API Gateway / Controllers
API Gateway / Controllers	Routing, request parsing, bearer-token extraction, JSON responses	Validation layer, AuthModule, all business modules
AuthModule	Credential check against a salted hash, token issue, LoginSession with 30-minute sliding inactivity expiry, RBAC checks	Database
UserModule	Create, edit and deactivate Teacher and Student accounts	AuthModule, Database
CourseModule	Courses, sections, schedules, Teacher assignment	Database
EnrollmentModule	Enroll, transfer and remove students (status flag, history kept); supplies the active roster	CourseModule, Database
AttendanceModule	Marks Present/Absent/Late; assignment and roster checks; stores metadata; correction-window edits; audit log; publishes AttendanceSubmitted / AttendanceEdited	EnrollmentModule, CourseModule, Database
ReportModule	Attendance percentage, defaulter list, CSV/PDF export; recalculates on attendance events and evaluates the threshold; raises LowAttendance	Database; consumes Attendance events

Component	Responsibility	Depends on
NotificationModule	Creates low-attendance alerts, handles leave requests and decisions, dispatches e-mail	Database, Email service; consumes Report events
Validation / Error Layer	Input validation and uniform error responses	None

3.5 Chosen Architecture Pattern and Rationale

SAMS uses a three-tier layered architecture (presentation, business, data). The browser communicates with the server through a REST API over HTTPS. The business tier follows the MVC pattern: views are the role-specific pages rendered by the UI Module; controllers are the REST controllers, and models are the business services and their relational entities.

Choice	Rationale
Layered (3-tier)	Separates UI, business rules and storage, so each layer can be tested independently, matching the unit and integration levels in the Test Plan
MVC in the business tier	Role-specific dashboards (views) reuse the same controllers and services
Modular monolith (over microservices)	A single deployable unit is sufficient for institutional scale (100 concurrent users) and keeps the attendance-plus-audit transaction simple. Microservices were considered and rejected as unnecessary operational overhead at this scale
Event hook (Observer) for the alert chain	Attendance submission or edit publishes AttendanceSubmitted / AttendanceEdited, consumed only by ReportModule for recalculation. ReportModule then raises LowAttendance, consumed only by NotificationModule (SAMS-F-014). Events run in-process and synchronously, after the attendance transaction commits and before the response is sent, so the alert exists in the same session; e-mail delivery is asynchronous and never blocks saving

3.6 Technology Stack and Data Stores

The design is technology neutral, as permitted by SRS Section 2.5. The following is assumed for sizing and the examples in Section 4:

Layer	Assumption
Web/application server	Any mainstream server-side web framework that supports a stateless REST controller layer and a database connection pool
Database	A relational database, chosen for strict referential integrity between courses, sections, enrollments and attendance, and for transactional guarantees (SAMS-NF-002)
Email / notification service	An outbound SMTP or transactional-email provider, called asynchronously by NotificationModule
Transport	TLS (HTTPS) between browser and server, as required by SRS Section 3.4

3.7 Risks and Mitigations

Risk	Mitigation
Network interruption or client timeout during attendance submission causes lost or duplicated records	Idempotency key per submission; the UI keeps the draft and retries with the same key (SAMS-NF-002)
Report or defaulter queries slow down as enrollment history grows	Indexes on AttendanceRecord, Session, Enrollment and LoginSession keep reads within the 5-second target (SAMS-NF-005)
Unauthorized or accidental edit of an attendance record	RBAC plus ownership checks, a configurable correction window, and an append-only audit log (SAMS-SR-002)

Risk	Mitigation
Email/notification service outage delays alerts	In-app notification is created first, inside the same transaction; e-mail is queued and retried, and never blocks the attendance save
Ambiguity between SRS and STP on who may edit attendance, and how a Late mark is counted	Resolved explicitly in Section 4.6 (Open Issues), with the more specific rule applied and flagged for confirmation

3.8 Traceability to Requirements

Requirement	Module / Design Element	Design Section
SAMS-F-001	AuthModule, LoginSession, login endpoint	3.9, 4.3, 4.4
SAMS-F-002	RBAC check on every endpoint	3.9, 4.3
SAMS-F-003	UserModule (soft deactivation)	4.3, 4.5
SAMS-F-004, F-005	CourseModule, SectionTeacher	4.3, 4.5
SAMS-F-006, F-007	EnrollmentModule (status flag, no delete), active roster	4.2, 4.3, 4.5
SAMS-F-008, F-009	AttendanceModule, Session and AttendanceRecord tables	4.2, 4.3, 4.5
SAMS-F-010	AttendanceModule edit path, AuditLog, configurable correction window	4.2, 4.3, 4.5, 4.6
SAMS-F-011	ReportModule, recalculation on attendance events	4.2, 4.3, 4.6
SAMS-F-012	ReportModule, Config threshold	4.2, 4.3, 4.5
SAMS-F-013	ReportModule export	4.3
SAMS-F-014	NotificationModule, event chain, Notification table	3.5, 4.2, 4.3, 4.5
SAMS-F-015	NotificationModule, LeaveRequest table	4.3, 4.5
SAMS-NF-001	Single-transaction batch insert; recalculation limited to affected students; indexed queries	3.6, 4.2, 4.5
SAMS-NF-002	Idempotency key, transactional save, draft kept in UI, retry after re-login	4.2, 4.3, 4.4, 4.5
SAMS-NF-003	Single-screen marking UI with bulk default actions	4.5, 4.6
SAMS-NF-004	Stateless controllers, database connection pool, staging load test	3.1, 3.6
SAMS-NF-005	Indexed report queries	3.7, 4.5
SAMS-NF-006	Structured logging, health checks, connection pool	3.7
SAMS-SR-001	Validation layer, parameterised queries	3.9, 4.2, 4.4
SAMS-SR-002	Marking-Teacher-or-Admin rule, correction window, AuditLog	3.9, 4.2, 4.3, 4.4
SAMS-SR-003	Admin-only endpoints (users, courses, config)	3.9, 4.3
SAMS-SR-004	RBAC plus ownership checks on read and write endpoints	3.9, 4.2, 4.3, 4.4
SAMS-SR-005	LoginSession sliding inactivity expiry	3.9, 4.3, 4.5
SO-001, SO-002	No hard deletes, soft deactivation, AuditLog	3.1, 3.9, 4.5

3.9 Security Architecture

Layer	Control	Requirement
Transport	HTTPS between browser and server	SRS 3.4
Authentication	ID and password; passwords stored as salted adaptive hashes; generic failure message	SAMS-F-001

Layer	Control	Requirement
Session	Server-side LoginSession with a 30-minute sliding inactivity expiry: every authenticated request updated last-activity; a request after 30 idle minutes is rejected with 401 and re-authentication is required. Logout revokes the session	SAMS-SR-005
Authorization	RBAC checks every endpoint. Ownership checks on both read and writes: a Teacher may access only sections listed in SectionTeacher for their user ID; a Student may access only their own ID; Admin has full access	F-002, SR-003, SR-004
Input	Server-side validation (type, length, whitelist); parameterised queries	SAMS-SR-001
Integrity	Only the marking Teacher (within the correction window) or an Admin can edit a record; every edit is written to AuditLog in the same transaction	SAMS-SR-002
Data protection	No hard deletes of attendance; deactivation is a flag; logs exclude passwords and personal data	SO-001, SO-002

Trust boundaries: (1) browser and server, (2) server and database, (3) server and email service. All input crossing boundary 1 is treated as untrusted.

4. Design

4.1 Design Overview

SAMS is designed as layered modules for separation of concerns and easier maintenance (Section 3.4). A request enters through the API Gateway, is authenticated and validated, and is then handled by one module, which uses the database through its own repository calls. Modules collaborate through in-process events (Section 3.5) rather than direct calls, so AttendanceModule, ReportModule and NotificationModule stay independently testable.

4.2 UML Sequence Diagrams

Three sequence diagrams cover the main workflows: marking attendance, the resulting percentage recalculation and low-attendance alert, and correcting a record within the window.

Mark Attendance (SAMS-F-008, F-009, NF-001, NF-002, SR-001, SR-004)

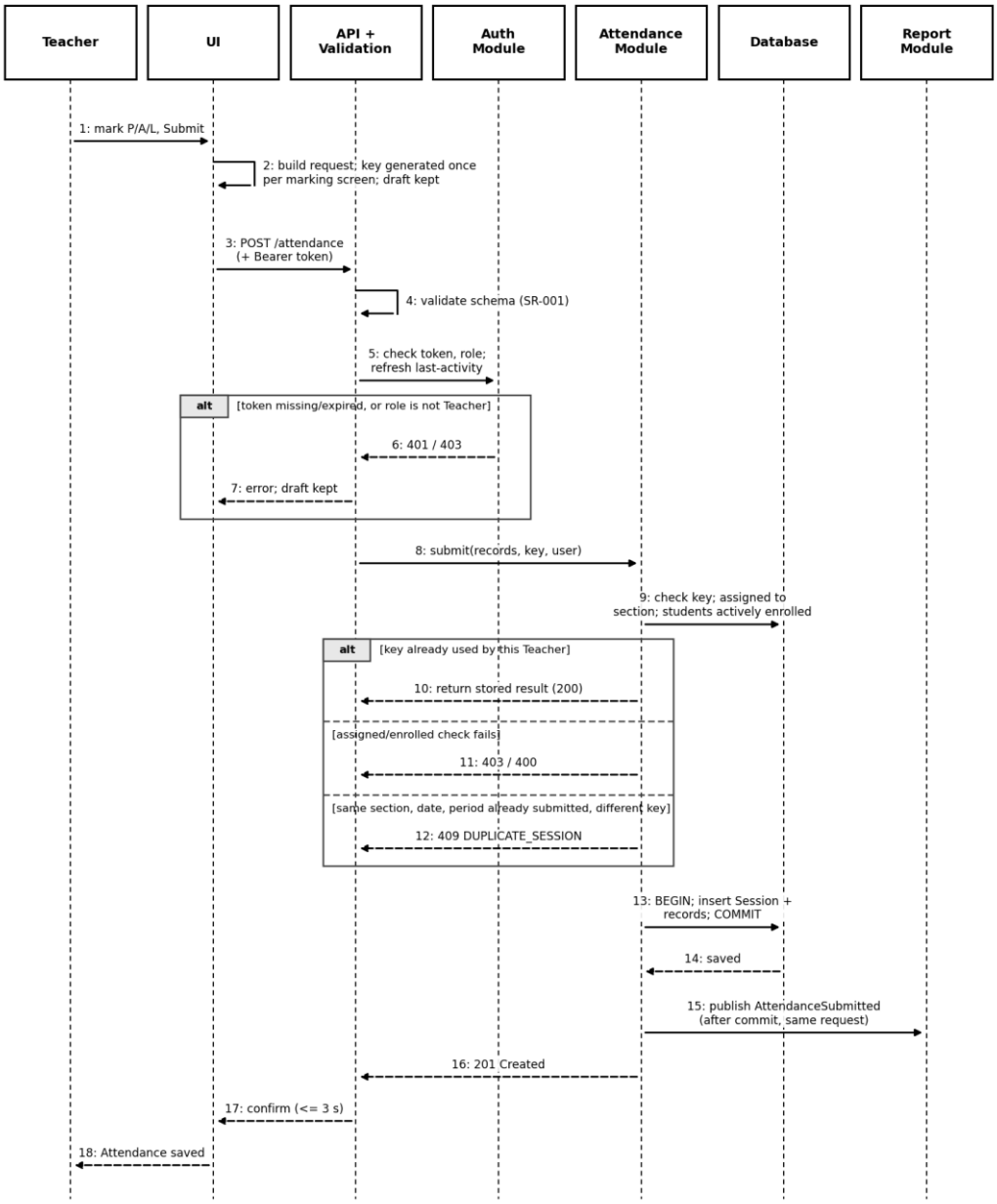


Figure 2: Sequence diagram, mark attendance

- **Ownership and roster checks (step 9):** the Teacher must be assigned to the section (SectionTeacher) and every student in the request must have an ACTIVE enrollment in it; otherwise, the request is rejected and nothing is stored.
- **Idempotency (steps 9 to 10):** if the same Teacher retries with the same key, for example after a timeout, the stored result is returned and no records are created. A different key for a section, date and period that already has a Session is a true duplicate and gets 409 DUPLICATE_SESSION.
- **Session timeout during submission (NF-002):** a 401 response does not discard the draft. The UI keeps the marks and the key, asks the Teacher to log in again, and resubmits the same key.

Recalculate Percentage and Low-Attendance Alert (SAMS-F-011, F-012, F-014)

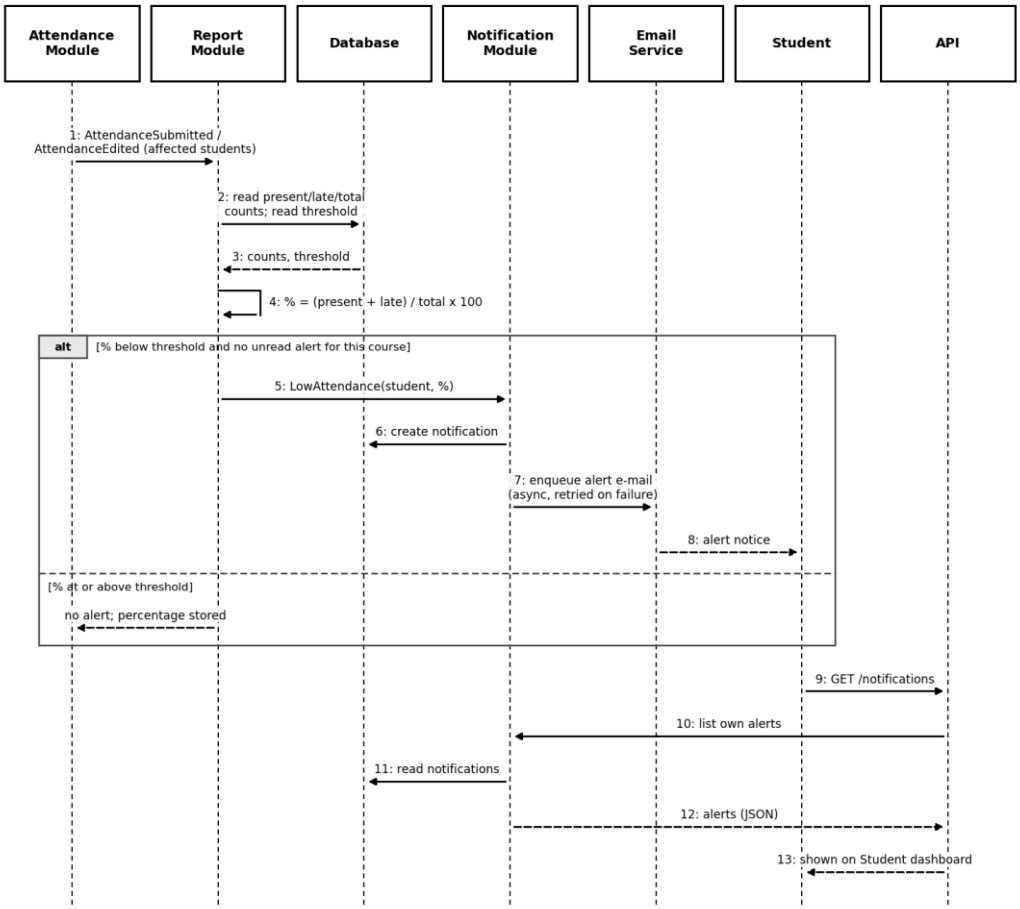


Figure 3: Sequence diagram, percentage recalculation and alert

- **Triggering:** the same flow runs after AttendanceEdited (see below), so percentages and alerts never go stale after a correction.
- **Alert suppression:** an alert is created only when the percentage is below the threshold, and the student has no unread low-attendance alert for that course, which prevents a new alert on every submission.

Edit Attendance in Correction Window (SAMS-F-010, SR-002)

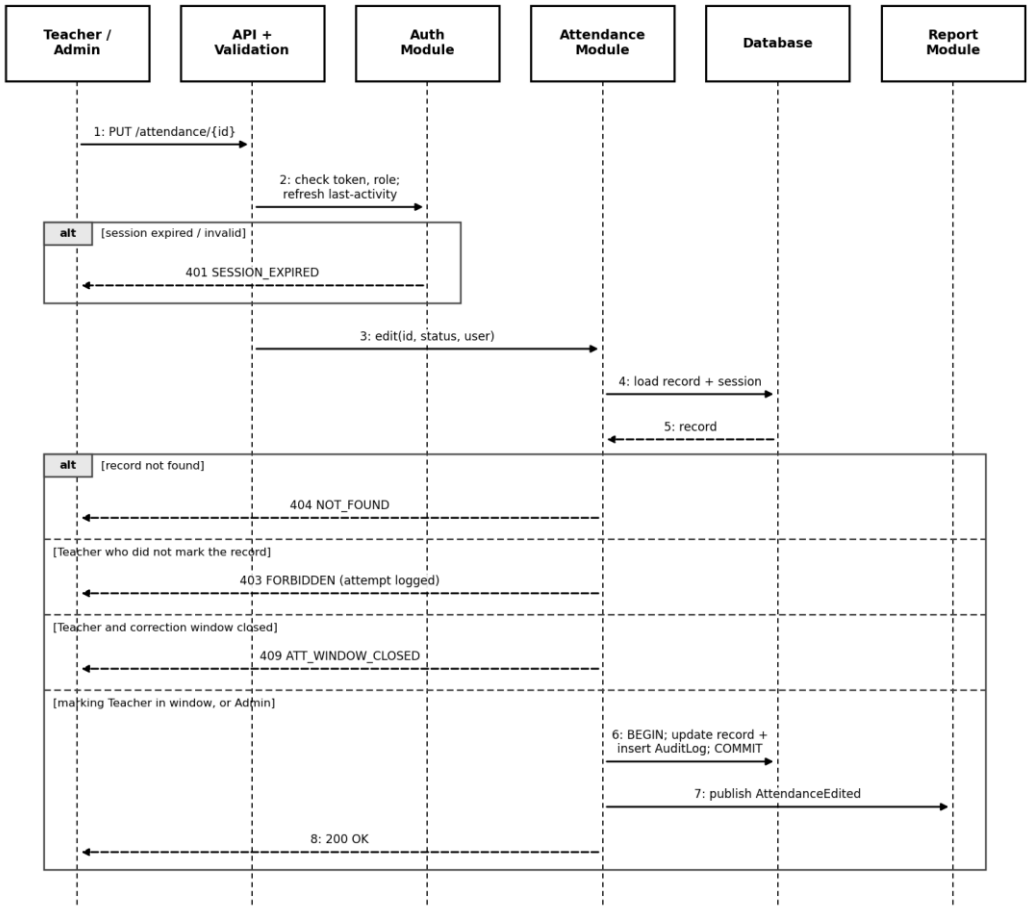


Figure 4: Sequence diagram, edit attendance

- **Who may edit:** a teacher may edit only records they marked, and only while the correction window is open. An Admin may edit any record and is not subject to the window; every edit is audited.
- **Atomicity:** the record update and the AuditLog insert commit in one transaction; if fails, both roll back.

4.3 API Design

Base path /api/v1. JSON over HTTPS. Bearer token in the Authorization header. Every authenticated call refreshes the login session's last-activity time.

Method and Path	Purpose	Role	Req
POST /auth/login	Login, returns token and role	All	F-001
POST /auth/logout	End session (revokes LoginSession)	All	F-001
GET / POST /users	List / create accounts	Admin	F-003
PATCH /users/{id}/deactivate	Deactivate account	Admin	F-003
GET / POST /courses	List / create courses	Admin; Teacher (own, GET)	F-004, F-005
POST /sections/{id}/teachers	Assign Teacher	Admin	F-005
POST /sections/{id}/students	Enroll student	Admin	F-006
PATCH /enrollments/{id}	Transfer or remove, keep history	Admin	F-007
GET /sections/{id}/roster	Active roster for marking	Teacher (own); Admin	F-006

Method and Path	Purpose	Role	Req
POST /attendance	Submit attendance for a session (idempotent)	Teacher (own)	F-008, F-009, NF-002
GET /sections/{id}/attendance	Records for a date and period	Teacher (own); Admin	F-008, F-010
PUT /attendance/{id}	Edit a record	Marking Teacher (in window); Admin	F-010, SR-002
GET /audit-log	Attendance edits history	Admin	F-010, SR-002
GET /students/{id}/attendance	Percentage per course and overall	Student (own); Teacher (own); Admin	F-011, SR-004
GET /sections/{id}/defaulters	Defaulter list by threshold	Teacher (own); Admin	F-012
GET /reports/export	Export CSV/PDF	Teacher (own); Admin	F-013
GET /notifications	Own alerts and leave decisions	Student (own)	F-014, F-015
POST / GET /leave-requests	Submit / list leave requests	Student (own); Teacher (own sections)	F-015
PATCH /leave-requests/{id}	Approve or reject	Teacher (own sections)	F-015
GET /config, PUT /config/threshold	Read / set threshold and correction window	Admin	SR-003

Example - submit attendance:

```
POST /api/v1/attendance
{ "sectionId": 12, "date": "2026-09-29", "period": 2,
  "idempotencyKey": "b7e1-...",
  "records": [ {"studentId": 101, "status": "P"},
               {"studentId": 102, "status": "A"} ] }

201 Created
{ "saved": 2, "sessionId": 884 }
```

A retry with the same key (for example after a timeout) returns the original result and creates nothing: 200 OK, { "saved": 2, "sessionId": 884, "replayed": true }.

4.4 Error Handling, Logging and Monitoring

All errors use one format: { "error": { "code": "ATT_WINDOW_CLOSED", "message": "...", "details": [] } }. No sensitive information (passwords, personal data) appears in any error message or log.

HTTP	Code	Situation	Behaviour
400	VALIDATION_ERROR	Invalid, too long or unsupported input (SR-001); student not actively enrolled	Reject; nothing stored; field-level message
401	AUTH_FAILED / SESSION_EXPIRED	Bad credentials, or 30 minutes of inactivity (SR-005)	Redirect to login; unsent marking data is kept and resubmitted with the same idempotency key after re-login
403	FORBIDDEN	Wrong role; section not assigned to the Teacher; Student ID differs from own ID; editing a record marked by another Teacher	Deny and log the attempt; original data unchanged
404	NOT_FOUND	Unknown record	Clear message
409	ATT_WINDOW_CLOSED	Teacher edits after the correction window	Reject; original unchanged
409	DUPLICATE_SESSION	A different submission already exists for the same section, date and period	Reject; original unchanged

HTTP	Code	Situation	Behaviour
500	INTERNAL_ERROR	Unexpected fault	Generic message to user; details in server log
503	SERVICE_UNAVAILABLE	Database or email service down	Retry prompt; email failures queued for retry

Monitoring: transaction failure rate, average attendance-submission time (against the 3-second target), and application uptime are tracked so that NF-001, NF-004 and NF-006 can be measured in practice, not only at test time.

4.5 UX Design

The Admin, Teacher and Student consoles share one layout and typography, so the system feels consistent across roles. Design choices aimed at the usability requirements in the SRS:

- **Marking screen (NF-003):** one screen shows the whole roster, with all students pre-set to Present and a one-click "mark all" action, so a first-time Teacher can mark 40 students within two minutes by changing only the exceptions.
- **Accessible defaults:** adequate font size and contrast, and clear focus order for keyboard use, on all three consoles.
- **Feedback on every action:** a visible confirmation after a successful submission, and an inline, specific error message (not a generic failure) when a request is rejected.
- **Responsive layout:** screens remain usable on a tablet, since Teachers may mark attendance away from a desktop.

4.6 Open Issues and Next Steps

Open issues

Item	SRS / STP text	Design decision
Who may edit an attendance record	F-010 says a teacher edits; SR-002 says the marking Teacher or an Admin	This design follows SR-002, the more specific security rule. Recommend aligning F-010 and the Test Plan wording
Whether a Late mark counts as Present	The SRS defines the percentage over sessions marked Present and does not say how Late is treated	Design assumption A-1: Late is counted as Present unless institutional policy says otherwise. Needs confirmation, and a Late-entry test case should be added to the STP
How a low-attendance alert is sent	F-014 says alerts are sent automatically; the SRS use-case diagram shows the Teacher sending them	Alerts are generated automatically by the system; no manual-send endpoint is provided

Next steps

- Confirm the Late-status assumption (A-1) with the course coordinator and update the SRS and STP accordingly.
- Select the implementation framework and database and re-check the technology assumptions in Section 3.6.
- Run a timed usability session with an actual Teacher account to confirm the two-minute marking target (NF-003).
- Add login rate limiting; not required by the SRS, but a low-cost addition against repeated password guessing.

- Carry out the load test described in Section 3.1 (100 concurrent users) on a staging environment before release.

5. Appendices

5.1 Glossary

Term	Meaning
RBAC	Role-based access control
Login session	A server-tracked authenticated session (stored as a LoginSession record), distinct from a class Session/Period
Idempotency key	A unique value generated by the UI for one marking submission. Repeating a request with the same key returns the original result instead of creating duplicates
Sliding expiry	A login session expires 30 minutes after the last authenticated request, not 30 minutes after login
Event	An in-process message published by one module and consumed by another, for example AttendanceSubmitted
API	Application programming interface
Defaulter	A student whose attendance percentage is below the configured threshold
Correction window	The period during which a submitted attendance record may still be edited by the marking Teacher

5.2 References

- Software Requirements Specification (SRS), Student Attendance Management System, Team 3, v1.0, 04-09-2026
- Software Test Plan (STP), Student Attendance Management System, Team 3, v1.0, 17-09-2026
- IEEE Std 1016-2009, Systems Design - Software Design Descriptions

5.3 Tools

Component, sequence and entity-relationship diagrams in this document were prepared with standard UML/diagramming tools, in the same spirit as draw.io or PlantUML. The API notation in Section 4.3 follows common REST/JSON documentation style, like Swagger/OpenAPI.